

在 AlloyDB Omni 使用資料欄引擎，加快數據分析查詢速度

來源：<https://codelabs.developers.google.com/alloydb-omni-columnar?hl=zh-tw>

步驟數：10

在本程式碼研究室中，您將瞭解如何在運算 VM 中部署 AlloyDB Omni、載入資料，以及使用 AlloyDB Columnar Engine 提升效能

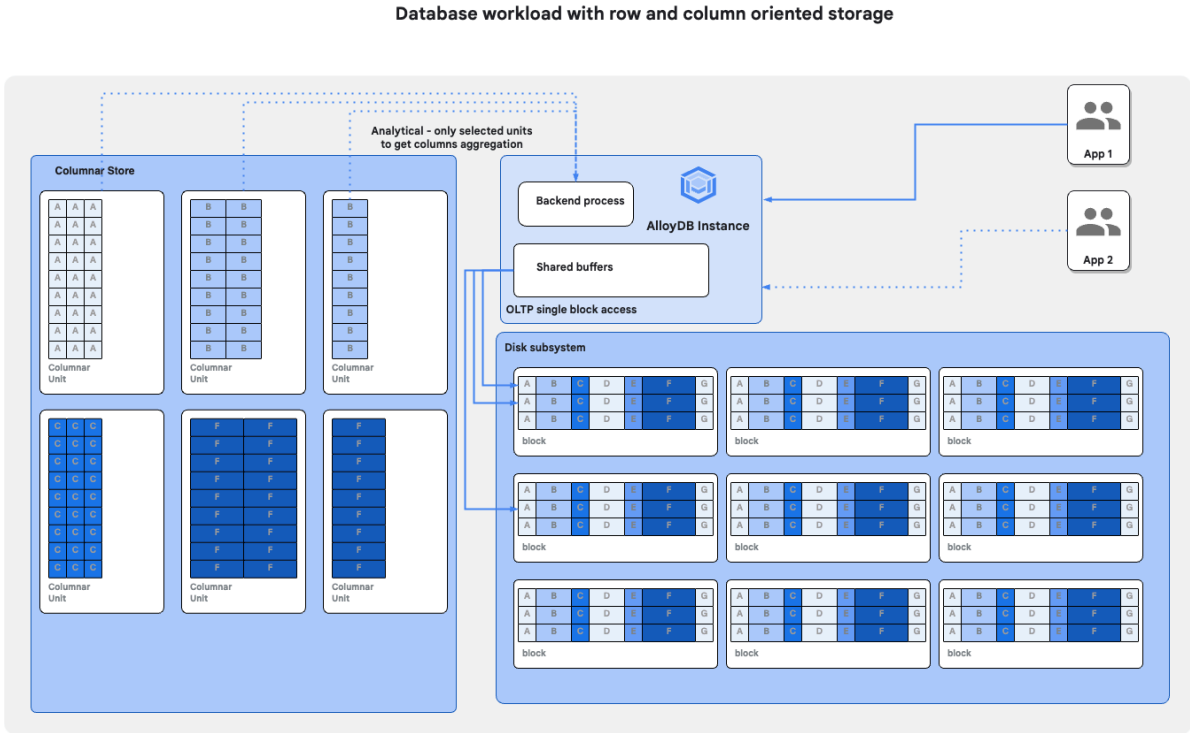
目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 事前準備](#)
4. [4. 在 GCE 上部署 AlloyDB Omni](#)
5. [5. 準備測試資料庫](#)
6. [6. 啟用資料欄引擎](#)
7. [7. 成效比較](#)
8. [8. 清除環境](#)
9. [9. 恭喜](#)
10. [10. 問卷調查](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將瞭解如何部署 AlloyDB Omni，並使用資料欄引擎提升查詢效能。



必要條件

- 對 Google Cloud 控制台有基本瞭解
- 指令列介面和 Google Shell 的基本技能

課程內容

- 如何在 Google Cloud 的 GCE VM 上部署 AlloyDB Omni
- 如何連線至 AlloyDB Omni
- 如何將資料載入 AlloyDB Omni
- 如何啟用資料欄引擎
- 如何檢查自動模式中的資料欄引擎
- 如何手動填入資料欄儲存庫

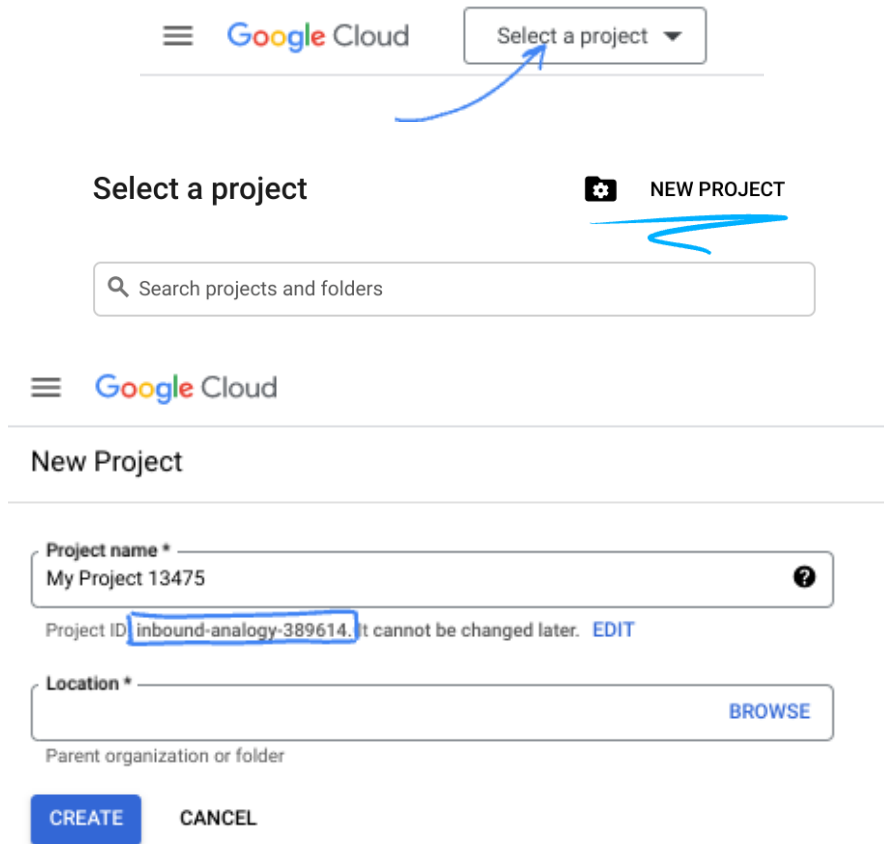
軟硬體需求

- Google Cloud 帳戶和 Google Cloud 專案
- 網路瀏覽器，例如 [Chrome](#)

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。



Google Cloud

Select a project ▼

Select a project

NEW PROJECT

Search projects and folders

Google Cloud

New Project

Project name *
My Project 13475

Project ID inbound-analogy-389614. It cannot be changed later. [EDIT](#)

Location * [BROWSE](#)

Parent organization or folder

CREATE CANCEL

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用。

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

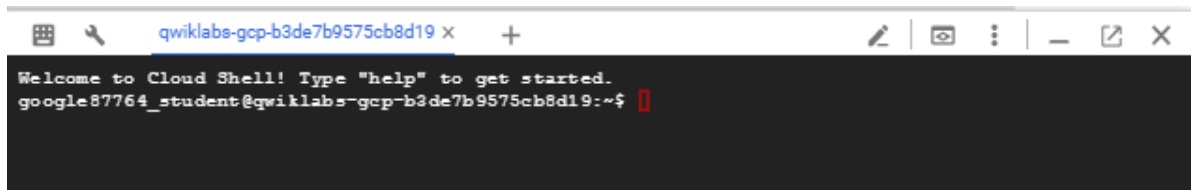
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：



這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

3. 事前準備

啟用 API

輸出內容：

請注意，如果您未使用促銷方案，啟用部分資源會產生費用。正常情況下，實驗室完成後所有資源都會遭到毀損，因此所有資源的費用不會超過 \$5 美元。建議您查看帳單，確認這項措施是否可接受。

在 Cloud Shell 中，確認專案 ID 已設定完畢：

```
PROJECT_ID=$(gcloud config get-value project)
echo $PROJECT_ID
```

如果未在 Cloud Shell 設定中定義，請使用下列指令進行設定

```
export PROJECT_ID=<your project>
gcloud config set project $PROJECT_ID
```

啟用所有必要服務：

```
gcloud services enable compute.googleapis.com
```

預期的輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ PROJECT_ID=$(gcloud config get-value
project)
Your active configuration is: [cloudshell-14650]
student@cloudshell:~ (test-project-001-402417)$ gcloud config set project test-project-
001-402417
Updated property [core/project].
student@cloudshell:~ (test-project-001-402417)$ gcloud services enable
compute.googleapis.com
Operation "operations/acat.p2-4470404856-1f44ebd8-894e-4356-bea7-b84165a57442" finished
successfully.
```

4. 在 GCE 上部署 AlloyDB Omni

如要在 GCE 上部署 AlloyDB Omni，我們需要準備具有相容設定和軟體的虛擬機器。以下範例說明如何在以 Debian 為基礎的 VM 上部署 AlloyDB Omni。

注意：如要查看最新版的 AlloyDB Omni 快速入門指南，請按[這裡](#)。

建立 GCE VM

我們需要部署 CPU、記憶體和儲存空間設定符合要求的 VM。我們將使用預設的 Debian 映像檔，並將系統磁碟大小增加至 20 GB，以容納 AlloyDB Omni 資料庫檔案。

我們可以透過啟動的 Cloud Shell 或已安裝 Cloud SDK 的終端機執行操作

AlloyDB Omni 的[快速入門](#)也說明瞭所有步驟。

為部署作業設定環境變數。

```
export ZONE=us-central1-a
export MACHINE_TYPE=n2-highmem-2
export DISK_SIZE=20
export MACHINE_NAME=omni01
```

接著，我們使用 gcloud 建立 GCE VM。

```
gcloud compute instances create $MACHINE_NAME \
  --project=$(gcloud info --format='value(config.project)') \
  --zone=$ZONE --machine-type=$MACHINE_TYPE \
  --metadata=enable-os-login=true \
  --create-disk=auto-delete=yes,boot=yes,size=$DISK_SIZE,image=projects/debian-
cloud/global/images/$(gcloud compute images list --filter="family=debian-12 AND family!=debian-12-
arm64" \
  --format="value(name)",type=pd-ssd
```

預期的控制台輸出內容：

```
gleb@cloudshell:~ (gleb-test-short-001-415614)$ export ZONE=us-centrall1-a
export MACHINE_TYPE=n2-highmem-2
export DISK_SIZE=20
export MACHINE_NAME=omni01
gleb@cloudshell:~ (gleb-test-short-001-415614)$ gcloud compute instances create
$MACHINE_NAME \
    --project=$(gcloud info --format='value(config.project)') \
    --zone=$ZONE --machine-type=$MACHINE_TYPE \
    --metadata=enable-os-login=true \
    --create-disk=auto-delete=yes,boot=yes,size=$DISK_SIZE,image=projects/debian-
cloud/global/images/$(gcloud compute images list --filter="family=debian-12 AND
family!=debian-12-arm64" \
    --format="value(name)"),type=pd-ssd
Created [https://www.googleapis.com/compute/v1/projects/gleb-test-short-001-
415614/zones/us-centrall1-a/instances/omni01].
WARNING: Some requests generated warnings:
  - Disk size: '20 GB' is larger than image size: '10 GB'. You might need to resize the
root repartition manually if the operating system does not support automatic resizing. See
https://cloud.google.com/compute/docs/disks/add-persistent-disk#resize_pd for details.

NAME: omni01
ZONE: us-centrall1-a
MACHINE_TYPE: n2-highmem-2
PREEMPTIBLE:
INTERNAL_IP: 10.128.0.3
EXTERNAL_IP: 35.232.157.123
STATUS: RUNNING
gleb@cloudshell:~ (gleb-test-short-001-415614)$
```

安裝 AlloyDB Omni

連線至建立的 VM：

```
gcloud compute ssh omni01 --zone $ZONE
```

預期的控制台輸出內容：


```
gleb@cloudshell:~ (gleb-test-short-001-415614)$ gcloud compute ssh omni01 --zone $ZONE
Warning: Permanently added 'compute.5615760774496706107' (ECDSA) to the list of known
hosts.
Linux omni01.us-centrall1-a.c.gleb-test-short-003-421517.internal 6.1.0-20-cloud-amd64 #1
SMP PREEMPT_DYNAMIC Debian 6.1.85-1 (2024-04-11) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
gleb@omni01:~$
```

在連線的終端機中執行下列指令。

在 VM 上安裝 Docker：

```
sudo apt update
sudo apt-get -y install docker.io
```

預期的控制台輸出內容(已遮蓋)：

```
gleb@omni01:~$ sudo apt update
sudo apt-get -y install docker.io
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [30 B]
Get:5 file:/etc/apt/mirrors/debian-security.list Mirrorlist [39 B]
Get:7 https://packages.cloud.google.com/apt google-compute-engine-bookworm-stable
InRelease [5146 B]
Get:8 https://packages.cloud.google.com/apt cloud-sdk-bookworm InRelease [6406 B]
Get:9 https://packages.cloud.google.com/apt google-compute-engine-bookworm-stable/main
amd64 Packages [1916 B]
Get:2 https://deb.debian.org/debian bookworm InRelease [151 kB]
...
Setting up binutils (2.40-2) ...
Setting up needrestart (3.6-4+deb12u1) ...
Processing triggers for man-db (2.11.2-2) ...
Processing triggers for libc-bin (2.36-9+deb12u4) ...
gleb@omni01:~$
```

定義 postgres 使用者的密碼：

```
export PGPASSWORD=<your password>
```

建立 AlloyDB Omni 資料的目錄。這是選用方法，但建議採用。根據預設，系統會使用 Docker 暫時檔案系統層建立資料，並在刪除 Docker 容器時一併刪除所有資料。將容器分開存放，可讓您獨立管理容器和資料，並視需要將容器放在 IO 特性較佳的儲存空間中

以下指令會在使用者主目錄中建立目錄，所有資料都會放在這個目錄中：

```
mkdir -p $HOME/alloydb-data
```

部署 AlloyDB Omni 容器：

注意：從存放區下載容器映像檔並啟動容器可能需要幾分鐘。

```
sudo docker run --name my-omni \  
-e POSTGRES_PASSWORD=$PGPASSWORD \  
-p 5432:5432 \  
-v $HOME/alloydb-data:/var/lib/postgresql/data \  
-v /dev/shm:/dev/shm \  
-d google/alloydbomni
```

預期的控制台輸出內容(已遮蓋)：

```
gleb@omni01:~$ export PGPASSWORD=StrongPassword  
gleb@omni01:~$ sudo docker run --name my-omni \  
-e POSTGRES_PASSWORD=$PGPASSWORD \  
-p 5432:5432 \  
-v $HOME/alloydb-data:/var/lib/postgresql/data \  
-v /dev/shm:/dev/shm \  
-d google/alloydbomni  
Unable to find image 'google/alloydbomni:latest' locally  
latest: Pulling from google/alloydbomni  
71215d55680c: Pull complete  
...  
2e0ec3fe1804: Pull complete  
Digest: sha256:d6b155ea4c7363ef99bf45a9dc988ce5467df5ae8cd3c0f269ae9652dd1982a6  
Status: Downloaded newer image for google/alloydbomni:latest  
56de4ae0018314093c8b048f69a1e9efe67c6c8117f44c8e1dc829a2d4666cd2  
gleb@omni01:~$
```

在 VM 中安裝 PostgreSQL 用戶端軟體 (選用 - 預期已安裝)：

```
sudo apt install -y postgresql-client
```

預期的控制台輸出內容：

```
gleb@omni01:~$ sudo apt install -y postgresql-client  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
postgresql-client is already the newest version (15+248).  
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
```

連線至 AlloyDB Omni：

```
psql -h localhost -U postgres
```

預期的控制台輸出內容：

```
gleb@omni01:~$ psql -h localhost -U postgres
psql (15.6 (Debian 15.6-0+deb12u1), server 15.5)
Type "help" for help.

postgres=#
```

中斷與 AlloyDB Omni 的連線：

```
exit
```

預期的控制台輸出內容：

```
postgres=# exit
gleb@omni01:~$
```

步驟 5 · 約 15 分鐘

5. 準備測試資料庫

如要測試資料欄引擎，我們需要建立資料庫，並填入一些測試資料。

建立資料庫

連線至 AlloyDB Omni VM 並建立資料庫

在 Cloud Shell 工作階段中執行下列指令：

注意：如果 Cloud Shell 工作階段已過期並停用，請重新設定專案並重設環境變數。

```
gcloud config set project $(gcloud config get-value project)
```

連線至 AlloyDB Omni VM：

```
ZONE=us-central1-a  
gcloud compute ssh omni01 --zone $ZONE
```

預期的控制台輸出內容：

```
student@cloudshell:~ (gleb-test-short-001-416213)$ gcloud config set project $(gcloud  
config get-value project)  
Updated property [core/project].  
student@cloudshell:~ (gleb-test-short-001-416213)$ ZONE=us-central1-a  
gcloud compute ssh omni01 --zone $ZONE  
Linux omni01.us-central1-a.c.gleb-test-short-003-421517.internal 6.1.0-20-cloud-amd64 #1  
SMP PREEMPT_DYNAMIC Debian 6.1.85-1 (2024-04-11) x86_64  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Mon Mar  4 18:17:55 2024 from 35.237.87.44  
student@omni01:~$
```

在建立的 SSH 工作階段中執行下列指令：

```
export PGPASSWORD=<your password>
```

```
psql -h localhost -U postgres -c "CREATE DATABASE quickstart_db"
```

預期的控制台輸出內容：

```
student@omni01:~$ psql -h localhost -U postgres -c "CREATE DATABASE quickstart_db"
CREATE DATABASE
student@omni01:~$
```

建立內含樣本資料的資料表

在測試中，我們將使用愛荷華州保險經紀人執照的公開資料。您可以在愛荷華州政府網站找到這個資料集：

https://data.iowa.gov/Regulation/Insurance-Producers-Licensed-in-Iowa/n4cc-vqyk/about_data。

首先，我們需要建立資料表。

在 GCE VM 中執行：

```
psql -h localhost -U postgres -d quickstart_db -c "DROP TABLE if exists
insurance_producers_licensed_in_iowa;
CREATE TABLE insurance_producers_licensed_in_iowa (
    npn int8,
    last_name text,
    first_name text,
    address_line_1 text,
    address_line_2 text,
    address_line_3 text,
    city text,
    state text,
    zip int4,
    firstactivatedate timestamp,
    expirydate timestamp,
    business_phone text,
    email text,
    physical_location text,
    iowaresident text,
    loa_has_crop text,
    loa_has_surety text,
    loa_has_ah text,
    loa_has_life text,
    loa_has_variable text,
    loa_has_personal_lines text,
    loa_has_credit text,
    loa_has_excess text,
    loa_has_property text,
    loa_has_casualty text,
    loa_has_reciprocal text
);"
```

預期的控制台輸出內容：

```
otochkin@omni01:~$ psql -h localhost -U postgres -d quickstart_db -c "DROP TABLE if exists
insurance_producers_licensed_in_iowa;
CREATE TABLE insurance_producers_licensed_in_iowa (
    npn int8,
    last_name text,
    first_name text,
    address_line_1 text,
    address_line_2 text,
    address_line_3 text,
    city text,
    state text,
    zip int4,
    firstactivatedate timestamp,
    expirydate timestamp,
    business_phone text,
    email text,
    physical_location text,
    iowaresident text,
    loa_has_crop text,
    loa_has_surety text,
    loa_has_ah text,
    loa_has_life text,
    loa_has_variable text,
    loa_has_personal_lines text,
    loa_has_credit text,
    loa_has_excess text,
    loa_has_property text,
    loa_has_casualty text,
    loa_has_reciprocal text
);"
NOTICE:  table "insurance_producers_licensed_in_iowa" does not exist, skipping
DROP TABLE
CREATE TABLE
otochkin@omni01:~$
```

將資料載入資料表。

在 GCE VM 中執行：

```
curl https://data.iowa.gov/api/views/n4cc-vqyk/rows.csv | psql -h localhost -U postgres -d
quickstart_db -c "\copy insurance_producers_licensed_in_iowa from stdin csv header"
```

預期的控制台輸出內容：

```
otochkin@omni01:~$ curl https://data.iowa.gov/api/views/n4cc-vqyk/rows.csv | psql -h
localhost -U postgres -d quickstart_db -c "\copy insurance_producers_licensed_in_iowa from
stdin csv header"
   % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                               Dload  Upload   Total   Spent    Left   Speed
100 39.3M    0 39.3M    0     0 1004k      0 --:--:--  0:00:40 --:--:-- 1028k
COPY 210898
otochkin@omni01:~$
```

我們已將 210898 筆保險業務員記錄載入資料庫，可以進行一些測試。

執行測試查詢

使用 psql 連線至 quickstart_db，並啟用計時功能，測量查詢的執行時間。

在 GCE VM 中執行：

```
psql -h localhost -U postgres -d quickstart_db
```

預期的控制台輸出內容：

```
student@omni01:~$ psql -h localhost -U postgres -d quickstart_db
psql (13.14 (Debian 13.14-0+deb11u1), server 15.5)
WARNING: psql major version 13, server major version 15.
         Some psql features might not work.
Type "help" for help.

quickstart_db=#
```

在 PSQL 工作階段中執行：

```
\timing
```

預期的控制台輸出內容：

```
quickstart_db=# \timing
Timing is on.
quickstart_db=#
```

請找出前 5 大城市，這些城市有銷售意外和健康保險的保險業務員，且執照效期至少還有 6 個月。

在 PSQL 工作階段中執行：

```
SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
```

注意：傳回的資料列數可能不同。來源資料集會反映最新上傳的資料。

預期的控制台輸出內容：

```
quickstart_db=# SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
   city      | count
-----+-----
TAMPA        | 1885
OMAHA        | 1656
KANSAS CITY  | 1279
AUSTIN       | 1254
MIAMI        | 1003
(5 rows)

Time: 94.965 ms
```

最好執行多次測試查詢，以取得可靠的執行時間。我們可以發現，傳回結果的平均時間約為 94 毫秒。在後續步驟中，我們將啟用 AlloyDB 資料欄引擎，看看是否能提升效能。

退出 psql 工作階段：

```
exit
```


步驟 6 · 約 20 分鐘

6. 啟用資料欄引擎

現在我們需要在 AlloyDB Omni 上啟用資料欄引擎。

更新 AlloyDB Omni 參數

我們需要將 AlloyDB Omni 的執行個體參數「google_columnar_engine.enabled」切換為「on」，並重新啟動。

更新 /var/alloydb/config 目錄中的 postgresql.conf，然後重新啟動執行個體。

在 GCE VM 中執行：

```
sudo docker exec my-omni /bin/bash -c "echo google_columnar_engine.enabled=true
>>/var/lib/postgresql/data/postgresql.conf"
sudo docker exec my-omni /bin/bash -c "echo
shared_preload_libraries=\ 'google_columnar_engine,google_job_scheduler,google_db_advisor,google_st
orage\ ' >>/var/lib/postgresql/data/postgresql.conf"
sudo docker stop my-omni
sudo docker start my-omni
```

預期的控制台輸出內容：

```
student@omni01:~$ sudo docker exec my-omni /bin/bash -c "echo
google_columnar_engine.enabled=true >>/var/lib/postgresql/data/postgresql.conf"
sudo docker exec my-omni /bin/bash -c "echo
shared_preload_libraries=\ 'google_columnar_engine,google_job_scheduler,google_db_advisor,g
oogle_storage\ ' >>/var/lib/postgresql/data/postgresql.conf"
sudo docker stop my-omni
sudo docker start my-omni
my-omni
my-omni
student@omni01:~$
```

驗證資料欄引擎

使用 psql 連線至資料庫，並驗證資料欄引擎。

連線至 AlloyDB Omni 資料庫

在 VM 的 SSH 工作階段中連線至資料庫：

```
psql -h localhost -U postgres -d quickstart_db -c "show google_columnar_engine.enabled"
```

指令應會顯示已啟用的資料欄引擎。

預期的控制台輸出內容：

```
student@omni01:~$ psql -h localhost -U postgres -d quickstart_db -c "show
google_columnar_engine.enabled"
 google_columnar_engine.enabled
-----
on
(1 row)
```

步驟 7 · 約 10 分鐘

7. 成效比較

現在可以填入資料欄引擎商店，並驗證效能。

自動填入資料欄儲存區

根據預設，這項工作每小時會執行一次。為避免等待，我們將縮短這段時間至 10 分鐘。

在 GCE VM 中執行：

```
sudo docker exec my-omni /bin/bash -c "echo
google_columnar_engine.auto_columnarization_schedule='EVERY 10 MINUTES\'
>>/var/lib/postgresql/data/postgresql.conf"
sudo docker stop my-omni
sudo docker start my-omni
```

預期輸出內容如下：

```
student@omni01:~$ sudo docker exec my-omni /bin/bash -c "echo
google_columnar_engine.auto_columnarization_schedule='EVERY 5 MINUTES\'
>>/var/lib/postgresql/data/postgresql.conf"
sudo docker stop my-omni
sudo docker start my-omni
my-omni
my-omni
student@omni01:~$
```

確認設定

在 GCE VM 中執行：

```
psql -h localhost -U postgres -d quickstart_db -c "show
google_columnar_engine.auto_columnarization_schedule;"
```

預期輸出內容：

```
student@omni01:~$ psql -h localhost -U postgres -d quickstart_db -c "show
google_columnar_engine.auto_columnarization_schedule;"
google_columnar_engine.auto_columnarization_schedule
-----
EVERY 10 MINUTES
(1 row)

student@omni01:~$
```

檢查 Columnar Store 中的物件。這個值應為空。

在 GCE VM 中執行：

```
psql -h localhost -U postgres -d quickstart_db -c "SELECT database_name, schema_name,
relation_name, column_name FROM g_columnar_recommended_columns;"
```

預期輸出內容：

```
student@omni01:~$ psql -h localhost -U postgres -d quickstart_db -c "SELECT database_name,
schema_name, relation_name, column_name FROM g_columnar_recommended_columns;"
 database_name | schema_name | relation_name | column_name
-----+-----+-----+-----
(0 rows)

student@omni01:~$
```

連線至資料庫，並多次執行稍早執行的相同查詢。

在 GCE VM 中執行：

```
psql -h localhost -U postgres -d quickstart_db
```

在 PSQL 工作階段中。

啟用時間設定

注意：您只需為每個 PSQL 工作階段啟用一次計時功能。再次執行「\timing」會停用這項功能。

```
\timing
```

重複執行查詢數次：

```
SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
```

預期輸出內容：

```
quickstart_db=# SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
```

city	count
TAMPA	1885
OMAHA	1656
KANSAS CITY	1279
AUSTIN	1254
MIAMI	1003

(5 rows)

Time: 94.289 ms

```
quickstart_db=# SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
```

city	count
TAMPA	1885
OMAHA	1656
KANSAS CITY	1279
AUSTIN	1254
MIAMI	1003

(5 rows)

Time: 94.608 ms

```
quickstart_db=#
```

等待 10 分鐘，然後檢查 columnar 商店是否已填入 insurance_producers_licensed_in_iowa 表格的資料欄。

```
SELECT database_name, schema_name, relation_name, column_name FROM g_columnar_recommended_columns;
```

預期輸出內容：

```
quickstart_db=# SELECT database_name, schema_name, relation_name, column_name FROM
g_columnar_recommended_columns;
```

database_name	schema_name	relation_name	column_name
quickstart_db	public	insurance_producers_licensed_in_iowa	city
quickstart_db	public	insurance_producers_licensed_in_iowa	expirydate
quickstart_db	public	insurance_producers_licensed_in_iowa	loa_has_ah

(3 rows)

Time: 0.643 ms

注意：如不想等待，可以使用 psql 指令 `SELECT google_columnar_engine_recommend();` 手動觸發自動欄狀化。

現在我們可以再次執行 `insurance_producers_licensed_in_iowa` 資料表的查詢，看看效能是否有所提升。

```
SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
```

預期輸出內容：

```
quickstart_db=# SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
   city      | count
-----+-----
 TAMPA      | 1885
 OMAHA      | 1656
 KANSAS CITY | 1279
 AUSTIN     | 1254
 MIAMI      | 1003
(5 rows)
```

Time: 14.380 ms

```
quickstart_db=# SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
   city      | count
-----+-----
 TAMPA      | 1885
 OMAHA      | 1656
 KANSAS CITY | 1279
 AUSTIN     | 1254
 MIAMI      | 1003
(5 rows)
```

Time: 13.279 ms

執行時間已從 94 毫秒降至 14 毫秒。如果沒有看到任何改善，請檢查 `g_columnar_columns` 檢視畫面，確認資料欄是否已成功填入資料欄儲存區。

```
SELECT relation_name,column_name,column_type,status,size_in_bytes from g_columnar_columns;
```

預期輸出內容：

```
quickstart_db=# select relation_name,column_name,column_type,status,size_in_bytes from
g_columnar_columns;
      relation_name      | column_name | column_type | status | size_in_bytes
-----+-----+-----+-----+-----
-
insurance_producers_licensed_in_iowa | city        | text        | Usable |        664231
insurance_producers_licensed_in_iowa | expirydate  | timestamp   | Usable |        212434
insurance_producers_licensed_in_iowa | loa_has_ah  | text        | Usable |        211734
(3 rows)
```

現在我們可以驗證查詢執行計畫是否使用資料欄引擎。

在 PSQL 工作階段中執行：

```
EXPLAIN (ANALYZE,SETTINGS,BUFFERS)
SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah ='Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
```

預期輸出內容：

```

quickstart_db=# EXPLAIN (ANALYZE,SETTINGS,BUFFERS)
SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah = 'Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;

QUERY PLAN
-----
---
Limit (cost=2279.72..2279.73 rows=5 width=17) (actual time=12.248..12.252 rows=5
loops=1)
  -> Sort (cost=2279.72..2292.91 rows=5277 width=17) (actual time=12.246..12.248 rows=5
loops=1)
    Sort Key: (count(*)) DESC
    Sort Method: top-N heapsort  Memory: 25kB
    -> HashAggregate (cost=2139.30..2192.07 rows=5277 width=17) (actual
time=10.235..11.250 rows=7555 loops=1)
      Group Key: city
      Batches: 1  Memory Usage: 1169kB
      -> Append (cost=20.00..1669.24 rows=94012 width=9) (actual
time=10.231..10.233 rows=94286 loops=1)
        -> Custom Scan (columnar scan) on
insurance_producers_licensed_in_iowa (cost=20.00..1665.22 rows=94011 width=9) (actual
time=10.229..10.231 rows=94286 loops=1)
          Filter: ((loa_has_ah = 'Yes'::text) AND (expirydate > (now() +
'6 mons'::interval)))
          Rows Removed by Columnar Filter: 116612
          Rows Aggregated by Columnar Scan: 94286
          Columnar cache search mode: native
          -> Seq Scan on insurance_producers_licensed_in_iowa
(cost=0.00..4.02 rows=1 width=9) (never executed)
            Filter: ((loa_has_ah = 'Yes'::text) AND (expirydate > (now() +
'6 mons'::interval)))
          Planning Time: 0.216 ms
          Execution Time: 12.353 ms

```

我們可以發現，系統從未對 `business_licenses` 資料表區隔執行「Seq Scan」作業，而是使用「Custom Scan (columnar scan)」。

這有助於我們將回應時間從 94 毫秒縮短至 12 毫秒。

如要清除資料欄引擎自動填入的內容，可以使用 SQL 函式 `google_columnar_engine_reset_recommendation`。

在 PSQL 工作階段中執行：

```
SELECT google_columnar_engine_reset_recommendation(drop_columns => true);
```

系統會清除已填入的資料欄，您可以在 `g_columnar_columns` 和 `g_columnar_recommended_columns` 檢視畫面中驗證，如先前所示。

```
SELECT database_name, schema_name, relation_name, column_name FROM g_columnar_recommended_columns;
```



```
SELECT relation_name,column_name,column_type,status,size_in_bytes from g_columnar_columns;
```

預期輸出內容：

```
quickstart_db=# SELECT database_name, schema_name, relation_name, column_name FROM
g_columnar_recommended_columns;
 database_name | schema_name | relation_name | column_name
-----+-----+-----+-----
(0 rows)

Time: 0.447 ms
quickstart_db=# select relation_name,column_name,column_type,status,size_in_bytes from
g_columnar_columns;
 relation_name | column_name | column_type | status | size_in_bytes
-----+-----+-----+-----+-----
(0 rows)

Time: 0.556 ms
quickstart_db=#
```

手動填入直欄式儲存空間

我們可以使用 SQL 函式手動將資料欄新增至 Columnar Engine Store，或在執行個體標記中指定必要實體，以便在執行個體啟動時自動載入這些實體。

使用 `google_columnar_engine_add` SQL 函式，加入與先前相同的資料欄。

在 PSQL 工作階段中執行：

```
SELECT google_columnar_engine_add(relation => 'insurance_producers_licensed_in_iowa', columns =>
'city,expirydate,loa_has_ah');
```

我們可以使用相同的 `g_columnar_columns` 檢視畫面驗證結果：

```
SELECT relation_name,column_name,column_type,status,size_in_bytes from g_columnar_columns;
```

預期輸出內容：

```
quickstart_db=# SELECT relation_name,column_name,column_type,status,size_in_bytes from
g_columnar_columns;
          relation_name          | column_name | column_type | status | size_in_bytes
-----+-----+-----+-----+-----
-
insurance_producers_licensed_in_iowa | city        | text        | Usable |        664231
insurance_producers_licensed_in_iowa | expirydate  | timestamp   | Usable |        212434
insurance_producers_licensed_in_iowa | loa_has_ah  | text        | Usable |        211734
(3 rows)

Time: 0.692 ms
quickstart_db=#
```

如要確認是否使用資料欄儲存空間，請執行與先前相同的查詢，並檢查執行計畫：

```
EXPLAIN (ANALYZE,SETTINGS,BUFFERS)
SELECT city, count(*)
FROM insurance_producers_licensed_in_iowa
WHERE loa_has_ah ='Yes' and expirydate > now() + interval '6 MONTH'
GROUP BY city ORDER BY count(*) desc limit 5;
```

結束 psql 工作階段：

```
exit
```

如果重新啟動 AlloyDB Omni 容器，我們會發現所有欄狀資訊都會遺失。

在殼層工作階段中執行下列指令：

```
sudo docker stop my-omni
sudo docker start my-omni
```

等待 5 到 10 秒，然後執行：

```
psql -h localhost -U postgres -d quickstart_db -c "SELECT
relation_name,column_name,column_type,status,size_in_bytes from g_columnar_columns"
```

預期輸出內容：

```
student@omni01:~$ psql -h localhost -U postgres -d quickstart_db -c "SELECT
relation_name,column_name,column_type,status,size_in_bytes from g_columnar_columns"
          relation_name          | column_name | column_type | status | size_in_bytes
-----+-----+-----+-----+-----
(0 rows)
```

如要在重新啟動時自動重新填入資料欄，我們可以將這些資料欄新增為 AlloyDB Omni 參數的資料庫標記。我們將新增旗標 `google_columnar_engine.relations='quickstart_db.public.insurance_producers_licensed_in_iowa(city,expirydate,loa_has_ah)'`，然後重新啟動容器。

在殼層工作階段中執行下列指令：

```
sudo docker exec my-omni /bin/bash -c "echo
google_columnar_engine.relations=\ 'quickstart_db.public.insurance_producers_licensed_in_iowa\
(city,expirydate,loa_has_ah\)\ ' >>/var/lib/postgresql/data/postgresql.conf"
sudo docker stop my-omni
sudo docker start my-omni
```

之後我們可以看到，啟動後，系統已自動將資料欄新增至 Columnar Store。

等待 5 到 10 秒，然後執行：

```
psql -h localhost -U postgres -d quickstart_db -c "SELECT
relation_name,column_name,column_type,status,size_in_bytes from g_columnar_columns"
```

預期輸出內容：

```
student@omni01:~$ psql -h localhost -U postgres -d quickstart_db -c "SELECT
relation_name,column_name,column_type,status,size_in_bytes from g_columnar_columns"
      relation_name      | column_name | column_type | status | size_in_bytes
-----+-----+-----+-----+-----
-
insurance_producers_licensed_in_iowa | city        | text        | Usable |        664231
insurance_producers_licensed_in_iowa | expirydate  | timestamp   | Usable |        212434
insurance_producers_licensed_in_iowa | loa_has_ah   | text        | Usable |        211734
(3 rows)
```

步驟 8 · 約 5 分鐘

8. 清除環境

現在我們可以終止 AlloyDB Omni VM

刪除 GCE VM

在 Cloud Shell 中執行下列指令：

```
export GCEVM=omni01
export ZONE=us-central1-a
gcloud compute instances delete $GCEVM \
  --zone=$ZONE \
  --quiet
```

預期的控制台輸出內容：

```
student@cloudshell:~ (test-project-001-402417)$ export GCEVM=omni01
export ZONE=us-central1-a
gcloud compute instances delete $GCEVM \
  --zone=$ZONE \
  --quiet
Deleted
```

步驟 9 · 約 1 分鐘

9. 恭喜

恭喜您完成本程式碼研究室。

涵蓋內容

- 如何在 Google Cloud 的 GCE VM 上部署 AlloyDB Omni
- 如何連線至 AlloyDB Omni
- 如何將資料載入 AlloyDB Omni
- 如何啟用資料欄引擎
- 如何檢查自動模式中的資料欄引擎
- 如何手動填入資料欄儲存庫

如要進一步瞭解如何使用資料欄引擎，請參閱[說明文件](#)。

步驟 10 · 約 1 分鐘

10. 問卷調查

輸出內容：
